

DATA CUT

Symphonie des données en flux continu

Rapport technique — Master 1 IA — École 89

Amaury Guindon · Intervenant : Najib AL AWAR · 2025–2026

Contexte & Problématique

Contexte

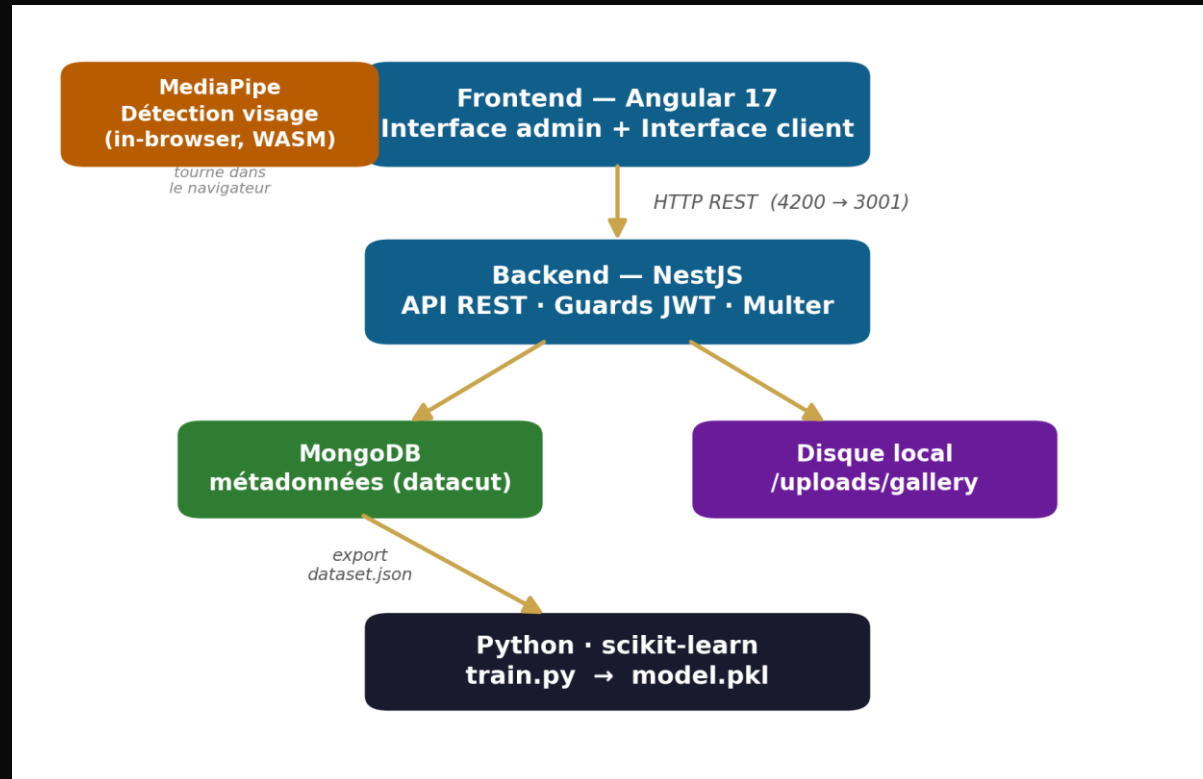
- Barbershop fictif exploitant ses photos de coupe
- Images labélisées : fade, afro, tresse, barbe...
- Besoin : pipeline structuré alimentant un modèle IA

Problématique

Comment collecter, labéliser et versionner des images tout en garantissant la traçabilité complète vers un modèle d'IA ?

Ingestion → Labélisation → Versioning → Entraînement → Recommandation

Architecture de la plateforme

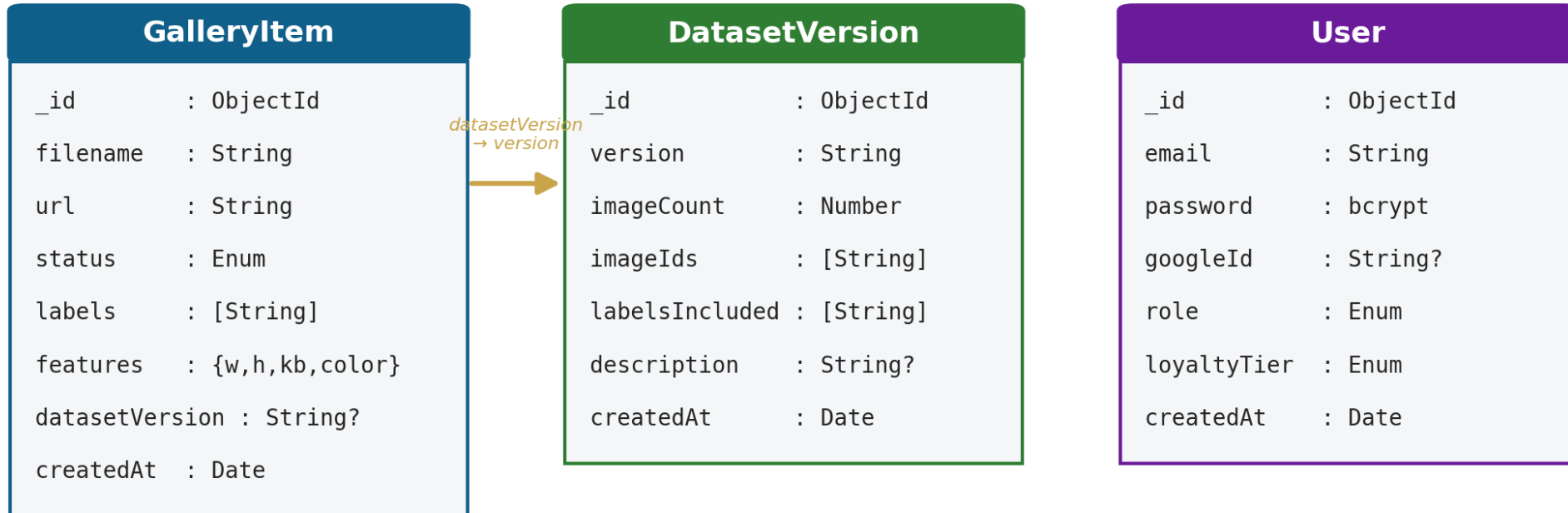


Séparation fichiers / métadonnées · ML offline + ML in-browser

Stack technique

Frontend	Angular 17 — standalone, signals
Backend	NestJS + Mongoose
Base de données	MongoDB (base datacut)
Feature extraction	sharp (Node.js)
ML entraînement	Python · scikit-learn · RandomForest
ML client	MediaPipe Face Landmarker (WebAssembly)
Authentification	JWT + OAuth2 Google

Schéma de base de données



Approche hybride : fichiers sur disque · métadonnées en MongoDB

GalleryItem.datasetVersion → DatasetVersion.version

Pipeline de données — 5 statuts



Transitions unidirectionnelles — traçabilité complète à chaque étape

- ▶ LABELED — 1 à N labels parmi 12 disponibles
- ▶ PROCESSED — validation finale avant export
- ▶ EXPORTED — image intégrée dans un snapshot versionné

Extraction automatique de features (sharp)

À chaque upload — automatique, transparent pour l'admin

width	Largeur en pixels	1080
height	Hauteur en pixels	1350
sizeKb	Taille du fichier	312 Ko
ratio	Largeur / Hauteur	0.80
r, g, b	Couleur dominante (RGB)	142, 98, 67

→ Vecteur 7D stocké en MongoDB · réutilisé directement par train.py

Versioning des datasets

Snapshots immuables · reproductibilité garantie

1

Créer une version

2

Sélection images
éligibles

3

Document
DatasetVersion

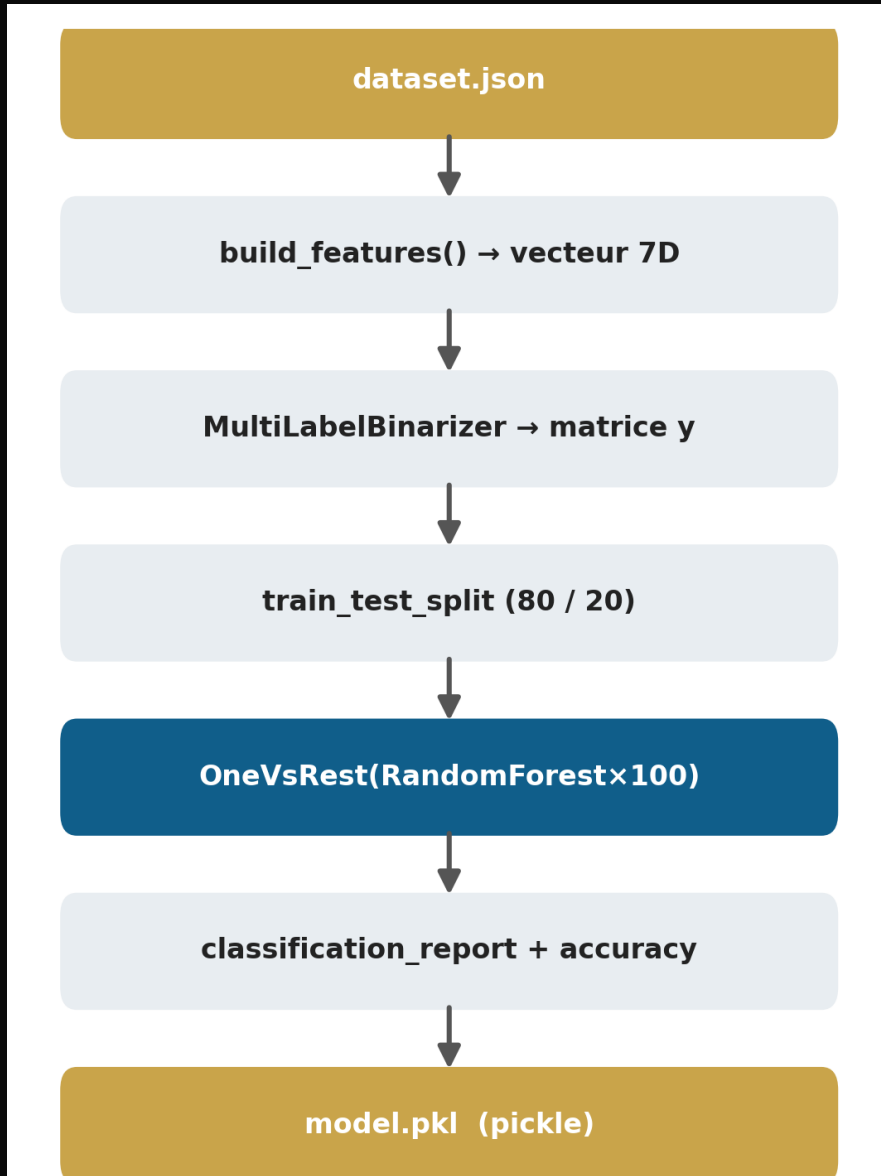
4

Export JSON

- Versions numérotées (v1, v2...) et horodatées
- Les images conservent la référence à leur version
- Relancer train.py sur le même JSON → résultat identique
- Historique complet des versions dans l'interface

Contrainte du projet respectée : versioning immuable ✓

Entraînement ML — train.py



Stratégie multi-label

OneVsRestClassifier permet à une image de porter plusieurs labels :

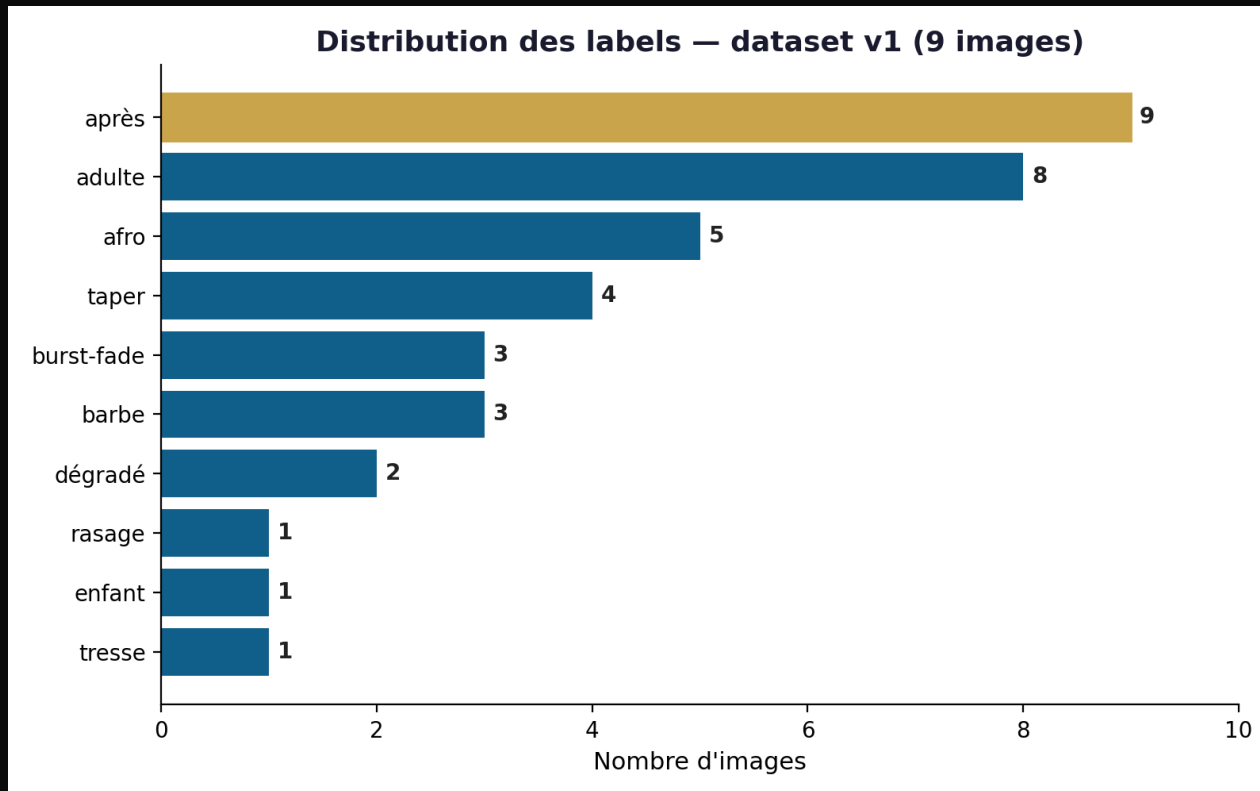
ex : fade + barbe + adulte

Paramètres

- `n_estimators = 100`
- `test_size = 0.2 (80/20)`
- `random_state = 42`

```
py train.py dataset-v1.json
```

Résultats & qualité du dataset



Métriques générées

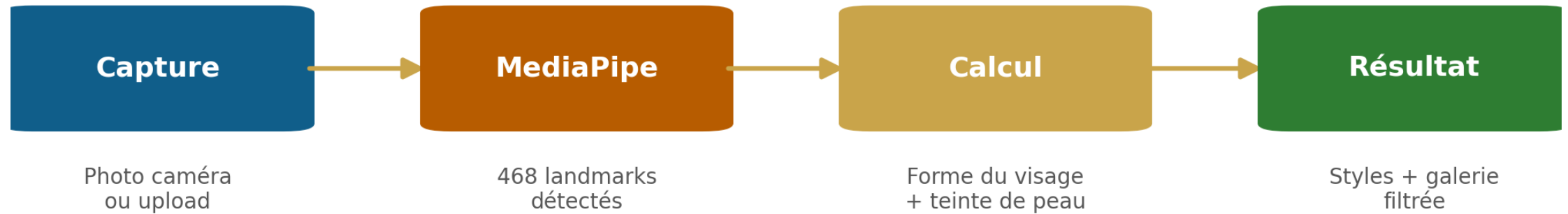
- Accuracy exacte
- Precision / Recall / F1 par label
- Support par label

Contexte

- Dataset prototype : 9 images
- Performances limitées par la taille
- Architecture scalable sans toucher au code

Plus de données → meilleur modèle

Recommandation client — « Ma Coupe Idéale »



Forme du visage

- Ovale → polyvalent
- Ronde → fade, taper
- Carrée → burst-fade, dégradé
- Cœur → dégradé, taper
- Oblongue → volume côtés

Teinte de peau

- Clair → styles lisses (fade, taper)
- Médium → tous les styles
- Foncé → afro, tresse, burst-fade

100 % dans le navigateur — aucune donnée biométrique transmise (RGPD by design)

Bonnes pratiques & gouvernance

✓ Séparation fichiers / métadonnées

Fichiers → disque · Métadonnées → MongoDB · migration S3/CDN possible

✓ Versioning immuable

Snapshots non modifiables · reproductibilité · auditabilité

✓ Sécurité & authentification

JWT + OAuth2 Google · rôles client/admin · Guards NestJS

✓ Protection des données (RGPD)

MediaPipe in-browser · aucune donnée biométrique transmise

Conclusion & Perspectives

Réalisé

- Pipeline complet (5 statuts)
- Features automatiques (sharp)
- Versioning immuable
- ML train.py → model.pkl
- Visualisation & supervision
- Recommandation IA (MediaPipe)
- Séparation fichiers / métadonnées

Perspectives

- 100+ images par label
- Stockage cloud (S3, Cloudinary)
- API REST exposant model.pkl
- Réentraînement automatique
- CNN / Vision Transformer

Architecture scalable : plus de données → meilleur modèle, sans modifier le code